No Cones, No Duals: A Fast and Scalable Solver for NRS*f*M

Supplementary Material

Abstract

This supplementary material provides additional experimental analysis, ablation studies, and theoretical details. Continuing from the main paper, we first present an experimental evaluation on the remaining benchmark datasets, including a statistical comparison with trajectory-space methods, which at present are not SOTA in terms of 3D reconstruction accuracy but are useful approaches. Next, we compare our approach's performance with recent deep learning methods on the CMU long-sequence dataset. Furthermore, the supplement presents essential results from the study, including what happens when we feed our rotation estimates to current state-of-the-art methods, as well as additional experimental observations on noisy data sequences and overall solver pipeline. On the theoretical side, we first provide a step-by-step derivation for Eq.(10) from the main paper. Next, extensive details regarding the complexity of the proposed NRSfM solver compared to the recent SOTA methods are presented. At the end, we prove the Lipschitz continuity of the reprojection error term in Eq.(11) and present additional qualitative results for completeness.

6. Additional Results on Benchmark Dataset

6.1. Comparison with trajectory-space approaches

Trajectory-space methods are important efficient alternatives to shape-space factorization. They are often fast because they reduce the number of unknowns through fixed temporal bases, trajectory priors, or probabilistic trajectory distributions, rather than solving for a full shape-space representation. Therefore, we compare them primarily in terms of reconstruction accuracy and representation assumptions. Our main runtime claim is restricted to recent shape-space SDP/ADMM baselines, where the proposed no-cone no-dual solver removes the dominant optimization bottlenecks.

***(i)* On CMU MoCap Dataset.** Trajectory space methods uses known DCT-basis to solve NRS*f*M [5, 20], thereby significantly reducing the number of unknowns to be solved. Therefore, by design these methods are generally faster in compute time than shape space methods, yet fails to model high-frequency details. Table 7 provides the e_{3d} statistical comparison with popular trajectory space approaches on CMU MoCap [5] and Torresani et al. [61] dataset. Based on the statistical results, it is easy to observe that more often than not our approach performs better.

***(ii)* Dense Face Dataset.** This dataset is composed of four face sequences with complex facial deformations. Table 8 provides the e_{3d} statistical comparison with popular trajectory space approaches. As stated, and can be observed from the statistical results that trajectory based approaches encounter challenges in modeling high-frequency details, whereas our approach performs well on all these sequences.

Data	CSF1 [20]	CSF2 [22]	KSTA [21]	PTA [5]	NP [63]	PBezier [2]	PPTA [4]	Ours
Drink	0.0223	0.0223	0.0156	0.0250	0.0354	**0.009**	0.011	0.0166
Pickup	0.2301	0.2277	0.2322	0.2369	0.0284	0.138	0.235	**0.0267**
Yoga	0.1467	0.1464	0.1476	0.1624	0.0210	0.111	0.158	**0.0148**
Stretch	0.0710	0.0685	0.0674	0.1088	0.0151	0.062	0.084	**0.0128**
Dance	0.2705	0.1983	0.2504	0.2960	**0.0980**	0.143	0.229	0.1218
Face	0.0363	0.0314	0.0339	0.0436	0.0352	–	–	**0.0179**
Walking	0.1893	0.1035	0.1029	0.3951	0.1223	–	–	**0.0592**
Shark	**0.0081**	0.0444	0.0160	0.1804	0.0942	–	–	0.0540

Table 7. e_{3d} comparison on MoCap datasets [5, 61] with trajectory-space and trajectory-prior NRSfM methods. Piecewise Bézier [2] and PPTA [4] are included as efficient trajectory-prior alternatives. Best performance for each sequence is highlighted in bold. PBezier and PPTA results are reported from their published papers because public code is unavailable; "–" denotes unavailable results.

Data	CSF1 [20]	CSF2 [22]	PTA [5]	NP [63]	Ours
Face Seq.1	0.5325	0.4677	0.1559	-	**0.0774**
Face Seq.2	0.9266	0.7909	0.1503	-	**0.0442**
Face Seq.3	0.5274	0.5474	0.1252	0.0394	**0.0382**
Face Seq.4	0.5392	0.5292	0.1348	0.0574	**0.0522**

Table 8. e_{3d} comparison on Dense Face Dataset with popular **trajectory space** [18]. Best performance for each sequence is highlighted in black.

6.2. More results on H3WB dataset

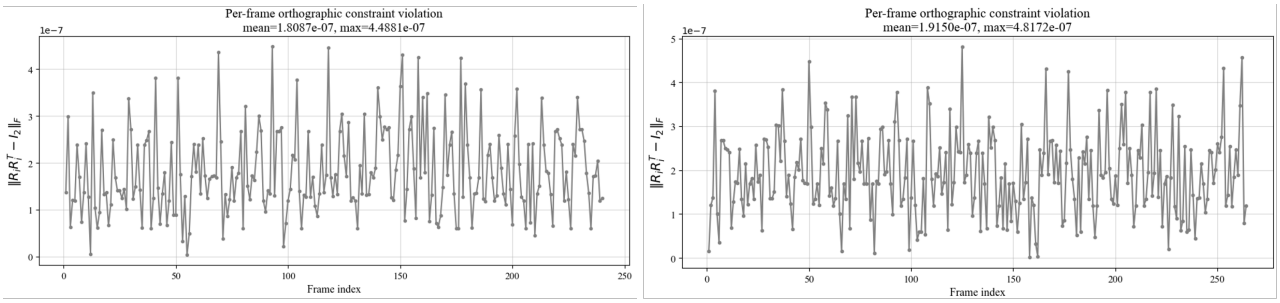
Continuing our evaluation on H3WB dataset [70] from the main paper, we further evaluated our approach on the other two camera motion provided with this dataset, i.e., Fixed-type camera motion and Multi-circle-type camera motion. Table 9 and Table 10 show e_{3d} comparison of our approach with the SOTA NRS*f*M matrix factorization approaches. In most of the cases for Fixed-Type camera motion, our approach achieves better e_{3d} while for Multi-circle-type camera motion our approach accuracy is favorably close of the SOTA method accuracy.

Fixed-type camera motion							
Data	CSF2 [22]	BMM [15]	PND [42]	CNS [43]	R-BMM [32]	TSM [57]	Ours
Eating2	0.2662	0.2084	0.2309	0.2250	0.1779	<u>0.1767</u>	**0.1671**
Smoking1	0.3534	0.2287	0.3397	0.2982	0.2290	**0.2040**	<u>0.2181</u>
Directions	0.2953	0.2872	0.3002	<u>0.2674</u>	0.2964	0.2688	**0.2149**
Smoking	0.1980	0.1759	0.4349	0.1968	0.1760	<u>0.1693</u>	**0.1664**
Waiting2	0.2125	0.1712	0.3306	0.1848	0.1478	**0.1097**	<u>0.1161</u>

Table 9. e_{3d} comparison on H3WB [70] with Fixed-type camera motion. 2nd best result is underlined.

6.3. Orthonormality residuals

For sequences without ground-truth camera rotations, direct rotation-error evaluation is not possible. We therefore assess



(a) Shark (b) Dance

Figure 7. Constraint-satisfaction analysis on CMU MoCap sequences without ground-truth rotations. For each frame, we report the row-orthonormality residual $\|\mathbf{R}_i\mathbf{R}_i^T - \mathbf{I}_2\|_F$ of the recovered 2×3 camera block after row-polar projection using Newton-Schulz inverse-square-root iterations. The plots show frame-wise residuals together with the sequence-level mean and maximum values. Across both the sequence, the residuals remain on the order of 10^{-7}, confirming that the recovered camera blocks satisfy the orthographic Stiefel constraint to high numerical precision.

Multi-circle-type camera motion							
Data	CSF2 [22]	BMM [15]	PND [42]	CNS [43]	R-BMM [32]	TSM [57]	Ours
Eating2	0.0658	0.0384	0.0395	0.0605	0.0335	**0.0276**	0.0333
Smoking1	0.0507	0.0486	0.0358	0.0662	0.0332	**0.0294**	0.0325
Directions	0.0456	0.0569	0.0322	0.0535	0.0518	**0.0315**	0.0438
Smoking	0.0708	0.0308	0.0292	0.0346	0.0328	0.0282	**0.0273**
Waiting2	0.0892	0.0742	0.0450	0.0873	0.0637	0.0390	0.0424

Table 10. e_{3d} comparison on H3WB [70] with Multi-circle-type camera motion. 2nd best result is underlined.

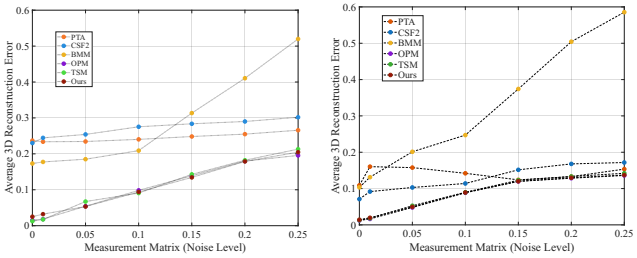
whether the recovered camera blocks satisfy the defining orthographic constraint. Specifically, for each frame, we compute $\|\mathbf{R}_i\mathbf{R}_i^T - \mathbf{I}_2\|_F$, where $\mathbf{R}_i \in \mathbb{R}^{2\times3}$ is the estimated camera block. As shown in Figure 7, on the Shark and Dance sequences, the residuals remain on the order of 10^{-7} across frames with similarly small mean and maximum errors. A similar trend is being observed on other dataset sequences as well. This confirms that the row-polar projection, implemented via Newton-Schulz inverse-square-root iterations, produces numerically stable row-orthonormal camera matrices even when ground-truth rotations are unavailable.

7. Comparison with deep-learning approaches on long-sequence CMU MoCap Dataset.

This large-scale dataset contains sequences from the CMU Motion Capture database, comprising 144 subjects, 31 keypoints, and more than $10,000$ frames. The train and test sets are divided as detailed in [29]. Since our approach does not rely on category-specific training, we compared it with existing methods on the test set (UNSEEN). Table 11 shows that our NRS*f*M solver despite learning any dataset driven object prior works well for large sequences, thus stretching the scope of factorization approach, which previously was restricted smaller sequences.

8. Additional Ablation Study

***(i)* Our solver rotation estimates to other methods.** We tested the suitability of the rotation matrix $\mathbf{R} \in \mathbb{R}^{2F\times3F}$ estimated using our approach when integrated with the existing SOTA approaches to recover 3D shape. Table 12 shows the e_{3d} achieved using it. Convincingly, the rotation recovered using our proposed solver help improve other method's 3D reconstruction result in most cases. For other cases, the difference in e_{3d} is in the order of 10^{-3}, thus insignificant, yet the compute time improvement can be significant *c.f.*, Table 2.



(a) e_{3d} on noisy Pickup Sequence (b) e_{3d} on noisy Stretch Sequence

Figure 8. e_{3d} variation w.r.t different noise levels in $\mathbf{W}$.

***(ii)* Additional results on noisy data sequence.** Figure 8 shows the performance of our approach on the Pickup and Stretch sequence compared to the other SOTA methods. Our approach shows a similar trend in the e_{3d} variations with different noise levels in $\mathbf{W}$. Such observations help us infer that our approach is robust in dealing with noise and well-suited for practical applications in soft robotics and intricate vision-based automation.

9. Detailed Pipeline

Figure 9 provides a detailed computational view of the proposed solver. Starting from the tracked measurement matrix $\mathbf{W}$, we first compute a low-rank SVD factorization and estimate the block-orthographic camera matrix $\mathbf{R}$ using the factor-side Euclidean correction described in Sec. 3.1 of the main paper. Given $\mathbf{R}$, the shape stage ini-

Methods	Subj. 07	Subj. 20	Subj. 23	Subj. 33	Subj. 34	Subj. 38	Subj. 39	Subj. 43	Subj. 93
C3DPO [49]	0.286	0.361	0.413	0.421	0.401	0.263	0.330	0.491	0.325
DNRSFM [29]	0.097	0.219	0.264	0.219	0.209	0.137	0.127	0.223	0.164
PR-PRN [66]	**0.061**	0.167	0.249	0.254	0.265	0.108	**0.028**	**0.080**	0.242
MHR-Net [67]	0.398	0.467	0.511	0.474	0.471	0.435	0.428	0.506	0.476
DNRSFM-R [17]	0.069	0.188	0.288	0.251	0.213	**0.096**	0.084	0.121	0.120
Ours	**0.061**	**0.141**	**0.194**	**0.196**	**0.192**	**0.099**	**0.072**	**0.092**	**0.097**

Table 11. e_{3d} on large CMU MOCAP (UNSEEN categories). Brown shows 2nd best.

Data	K-value	BMM [15]	R-BMM [32]	OPM [33]	TSM [57]
Drink	11	**0.0164** 0.0266	**0.0084** 0.0119	0.0083 0.0071	0.0041 0.0033
Pickup	12	**0.0321** 0.1731	**0.0164** 0.0198	0.0155 0.0152	0.0128 0.0126
Yoga	11	**0.0284** 0.1150	**0.0122** 0.0129	0.0140 0.0122	**0.0114** 0.0118
Stretch	13	**0.0294** 0.1034	**0.0126** 0.0144	**0.0116** 0.0124	0.0139 0.0131

Table 12. Suitability of our rotation solver when used in other high-performance methods. Bold indicate improved e_{3d} using our rotation estimates. For each method, **Top row** shows result using our rotation and **Bottom row** shows results reported by these methods using their rotation estimates.

tializes $\mathbf{S}^0 = \text{pinv}(\mathbf{R})\mathbf{W}$ and then applies the unrolled no-dual spectral-proximal solver. Each layer consists of a reprojection-gradient step, reshaping to the shape-space matrix $\mathbf{S}^\sharp$, weighted singular-value shrinkage, inverse reshaping, FISTA-style restart, and continuation. Thus, the full pipeline replaces SDP cone solvers for rotation and ADMM dual-variable updates for shape with a sequence of matrix multiplications, SVD-based shrinkage, and small row-polar projections.

10. Why the row–polar projection is the closest 2×3 row–orthonormal matrix?

Let's rewrite the original objective:

$$\arg\min_k \frac{1}{F}\sum_{i=1}^{F}\left(\left(\|\mathbf{g}_{1,ik}\|_2^2 - \|\mathbf{g}_{2,ik}\|_2^2\right)^2 + \left(\mathbf{g}_{1,ik}\cdot\mathbf{g}_{2,ik}\right)^2\right) \tag{15}$$

In essence, this equation is trying to measure how close the two vectors $\mathbf{g}_{1,ik}$ and $\mathbf{g}_{2,ik}$ are to being orthonormal. So, Eq.(15) selects the best index k^* such that the corresponding $\mathbf{\Omega}_{ik^*} \triangleq [\mathbf{g}_{1,ik^*}; \mathbf{g}_{2,ik^*}]$ is most orthonormal. Once we selected the best one, we want to make it strictly orthonormal. That's where the notion of exact projection is invoked, which is derived next.

Given a 2×3 matrix $\mathbf{A} = \mathbf{\Omega}_{ik^*}$ of row rank $r \leq 2$, we want the closest matrix $\mathbf{\Omega}$ whose rows are orthonormal, i.e., $\mathbf{\Omega} \in \text{St}(2,3) = \{\mathbf{\Omega} \in \mathbb{R}^{2\times 3} : \mathbf{\Omega}\mathbf{\Omega}^T = \mathbf{I}_2\}$ and

$$\mathbf{\Omega}^\star \in \min_{\mathbf{\Omega}\in\text{St}(2,3)} \|\mathbf{A} - \mathbf{\Omega}\|_F^2. \tag{16}$$

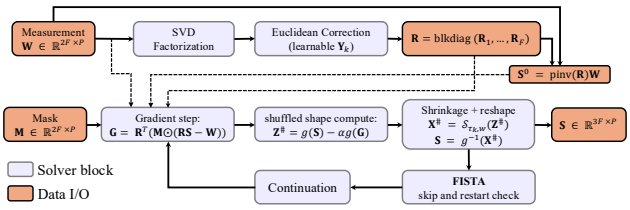


Figure 9. Detailed computational pipeline of the proposed no-cone, no-dual approach. The rotation stage performs factor-side Euclidean correction using learnable per-sequence factors $\mathbf{Y}_k$ and row-polar projection to recover valid camera blocks. The shape stage applies a fixed unrolled spectral-proximal loop consisting of reprojection-gradient updates, reshaping to $\mathbf{S}^\sharp$, weighted singular-value shrinkage, FISTA-style restart, and continuation. The main computations are matrix multiplications, SVD, shrinkage, and small row-polar projections.

Here, we show that the unique minimizer for full row rank is the row–polar factor

$$\mathbf{\Omega}^\star = (\mathbf{A}\mathbf{A}^T)^{-\frac{1}{2}}\mathbf{A} \tag{17}$$

which is exactly the expression used in the main text for $\mathbf{R}_i$:

$$\mathbf{R}_i = \left(\mathbf{\Omega}_{ik^\star}\mathbf{\Omega}_{ik^\star}^T\right)^{-\frac{1}{2}}\mathbf{\Omega}_{ik^\star}. \tag{18}$$

We also show that $\mathbf{\Omega}^\star$ is a partial isometry, i.e., row–orthonormal, and that it is the closest point in Frobenius norm on the Stiefel manifold.

10.1. Equivalent trace maximization

Expand the Frobenius objective Eq.(10):

$$\|\mathbf{A} - \mathbf{\Omega}\|_F^2 = \|\mathbf{A}\|_F^2 + \|\mathbf{\Omega}\|_F^2 - 2\mathbf{Tr}(\mathbf{\Omega}\mathbf{A}^T).$$

For any $\mathbf{\Omega} \in \text{St}(2,3)$, $\|\mathbf{\Omega}\|_F^2 = \mathbf{Tr}(\mathbf{\Omega}\mathbf{\Omega}^T) = \text{Tr}(\mathbf{I}_2) = 2$, therefore,

$$\|\mathbf{A} - \mathbf{\Omega}\|_F^2 = \|\mathbf{A}\|_F^2 + 2 - 2\mathbf{Tr}(\mathbf{\Omega}\mathbf{A}^T).$$

Minimizing $\|\mathbf{A} - \mathbf{\Omega}\|_F^2$ over $\mathbf{\Omega} \in \text{St}(2,3)$ is therefore equivalent to maximizing the linear functional

$$\mathbf{\Omega}^\star \in \arg\max_{\mathbf{\Omega}\in\text{St}(2,3)} \text{Tr}(\mathbf{\Omega}\mathbf{A}^T). \tag{19}$$

10.2. SVD reduction and solution form

Let the SVD of $\mathbf{A}$ be

$$\begin{aligned}&\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T, \text{ where, } \mathbf{U} \in \mathbb{R}^{2\times 2}, \ \mathbf{V} \in \mathbb{R}^{3\times 3}, \text{ and,}\\ &\mathbf{\Sigma} = \begin{bmatrix}\sigma_1 & 0 & 0\\ 0 & \sigma_2 & 0\end{bmatrix} \in \mathbb{R}^{2\times 3}, \ \ \sigma_1 \geq \sigma_2 \geq 0,\end{aligned} \tag{20}$$

so that $\mathbf{\Sigma}$ has possibly zero third column. Let's write

$$\mathbf{Tr}(\mathbf{\Omega}\mathbf{A}^T) = \mathbf{Tr}(\mathbf{\Omega}\mathbf{V}\mathbf{\Sigma}^T\mathbf{U}^T) = \mathbf{Tr}(\mathbf{U}^T\mathbf{\Omega}\mathbf{V}\mathbf{\Sigma}^T).$$

By defining the change of variables

$$\mathbf{B} \triangleq \mathbf{U}^T\mathbf{\Omega}\mathbf{V} \in \mathbb{R}^{2\times 3}.$$

Since $\mathbf{\Omega} \in \mathrm{St}(2,3)$, one has

$$\begin{aligned}\mathbf{B}\mathbf{B}^T &= (\mathbf{U}^T\mathbf{\Omega}\mathbf{V})(\mathbf{U}^T\mathbf{\Omega}\mathbf{V})^T\\ &= \mathbf{U}^T\mathbf{\Omega}\mathbf{V}\mathbf{V}^T\mathbf{\Omega}^T\mathbf{U}\\ &= \mathbf{U}^T(\mathbf{\Omega}\mathbf{\Omega}^T)\mathbf{U}\\ &= \mathbf{U}^T(\mathbf{I}_2)\mathbf{U}\\ &= \mathbf{I}_2\end{aligned} \tag{21}$$

i.e., $\mathbf{B} \in \mathrm{St}(2,3)$ as well. The objective (19) reduces to

$$\mathbf{Tr}(\mathbf{\Omega}\mathbf{A}^T) = \mathbf{Tr}(\mathbf{B}\mathbf{\Sigma}^T) = \sigma_1 B_{11} + \sigma_2 B_{22},$$

since $\mathbf{\Sigma}^T$ has entries (σ_1, σ_2) in positions $(1,1)$ and $(2,2)$ and zeros elsewhere. The constraints $\mathbf{B}\mathbf{B}^T = \mathbf{I}_2$ imply that the two rows of $\mathbf{B}$ are orthonormal in $\mathbb{R}^3$; in particular, $|B_{11}| \leq 1$ and $|B_{22}| \leq 1$ with orthogonality between rows. Since $\sigma_1 \geq \sigma_2 \geq 0$, the maximal value $\sigma_1 + \sigma_2$ is achieved by taking

$$\mathbf{B}^\star = \begin{bmatrix}\mathbf{I}_2 & \mathbf{0}_{2\times 1}\end{bmatrix},$$

for which $\mathbf{B}^\star(\mathbf{B}^\star)^T = \mathbf{I}_2$ and $\mathrm{tr}(\mathbf{B}^\star\mathbf{\Sigma}^T) = \sigma_1 + \sigma_2$. Thus, pulling back via $\mathbf{\Omega} = \mathbf{U}\mathbf{B}\mathbf{V}^T$,

$$\mathbf{\Omega}^\star = \mathbf{U}\begin{bmatrix}\mathbf{I}_2 & \mathbf{0}\end{bmatrix}\mathbf{V}^T \tag{22}$$

This is the unique maximizer when $\sigma_1 > \sigma_2 > 0$ (full row rank 2). In the rank–1 or isotropic case there is a low–dimensional family of maximizers yet it will still be captured by the polar factor detailed below.

10.3. Equivalence to the row polar factor

Here, we show that the formulation Eq.(10) proposed in the main paper to obtain the closest orthonormal 2×3 $\mathbf{R}_i$ is optimal and well-grounded theoretically. Assume the row–polar decomposition of a full–row–rank say $\mathbf{A} \in \mathbb{R}^{2\times 3}$, then we can write

$$\begin{aligned}&\mathbf{A} = \mathbf{H}\mathbf{\Omega},\\ &\text{where, } \mathbf{\Omega} \in \mathrm{St}(2,3), \text{ and, } \mathbf{H} = (\mathbf{A}\mathbf{A}^T)^{\frac{1}{2}} \in \mathbb{R}^{2\times 2},\end{aligned} \tag{23}$$

and the Stiefel factor is given explicitly by

$$\mathbf{\Omega} = \mathbf{H}^{-1}\mathbf{A} = (\mathbf{A}\mathbf{A}^T)^{-\frac{1}{2}}\mathbf{A}. \tag{24}$$

To show this is indeed the optimal form let's first compute with the SVD of $\mathbf{A}\mathbf{A}^T$

$$\begin{aligned}\mathbf{A}\mathbf{A}^T &= \mathbf{U}\mathbf{\Sigma}\mathbf{\Sigma}^T\mathbf{U}^T = \mathbf{U}\mathbf{diag}(\sigma_1^2, \sigma_2^2)\mathbf{U}^T,\\ \implies (\mathbf{A}\mathbf{A}^T)^{-\frac{1}{2}} &= \mathbf{U}\mathbf{diag}(\sigma_1^{-1}, \sigma_2^{-1})\mathbf{U}^T,\end{aligned} \tag{25}$$

Now if we multiply by $\mathbf{A}$ to the above expression so that we meet the form in Eq.(24), we get

$$\begin{aligned}(\mathbf{A}\mathbf{A}^T)^{-\frac{1}{2}}\mathbf{A} &= \mathbf{U}\mathbf{diag}(\sigma_1^{-1}, \sigma_2^{-1})\mathbf{U}^T(\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T)\\ &= \mathbf{U}\begin{bmatrix}\mathbf{I}_2 & 0\end{bmatrix}\mathbf{V}^T,\end{aligned} \tag{26}$$

which matches exactly Eq.(22), demonstrating the proposed expression as a unique minimizer.

Row–orthonormality. By construction the optimal $\mathbf{\Omega}$, symbolized as $\mathbf{\Omega}^\star$

$$\mathbf{\Omega}^\star(\mathbf{\Omega}^\star)^T = (\mathbf{A}\mathbf{A}^T)^{-\frac{1}{2}}\,\mathbf{A}\mathbf{A}^T(\mathbf{A}\mathbf{A}^T)^{-\frac{1}{2}} = \mathbf{I}_2.$$

Therefore, $\mathbf{\Omega}^\star \in \mathrm{St}(2,3)$.

Optimality and uniqueness. From the trace–maximization viewpoint as defined above,

$$\|\mathbf{A} - \mathbf{\Omega}\|_F^2 = \|\mathbf{A}\|_F^2 + 2 - 2\mathbf{Tr}(\mathbf{\Omega}\mathbf{A}^T),$$

so $\mathbf{\Omega}^\star$ maximizes $\mathbf{Tr}(\mathbf{\Omega}\mathbf{A}^T)$ and thus minimizes the cost function. If $\mathbf{A}$ has full row rank, the solution is unique. If $\mathbf{A}$ is rank–deficient, the same formula Eq.(24) holds with the inverse derived from the Moore–Penrose inverse of $(\mathbf{A}\mathbf{A}^T)^{\frac{1}{2}}$, the set of closest $\mathbf{\Omega}$ is not unique in that case yet the polar factor is one canonical choice.

10.4. Usage in our rotation estimation

In the rotation estimation approach, for each frame i we synthesize a 2×3 block $\mathbf{\Omega}_{ik^\star}$ from the learned corrective factor associated with basis $k^\star$. The closest valid camera block $\mathbf{R}_i$ with orthonormal rows is then obtained by the row–polar projection as :

$$\mathbf{R}_i = \arg\min_{\mathbf{\Omega}\in\mathrm{St}(2,3)} \|\mathbf{\Omega}_{ik^\star} - \mathbf{\Omega}\|_F^2 = \left(\mathbf{\Omega}_{ik^\star}\mathbf{\Omega}_{ik^\star}^T\right)^{-\frac{1}{2}}\mathbf{\Omega}_{ik^\star} \tag{27}$$

which the minimizer as shown above. This projection is differentiable almost everywhere (non-smooth only when $\mathbf{\Omega}_{ik^\star}$ is rank–deficient) and extremely fast to compute, making it ideal for a NRS*f*M rotation estimation.

Geometric intuition. The Stiefel manifold $\mathrm{St}(2,3)$ is the set of all 2–frames in $\mathbb{R}^3$, where a 2-frame consists of two orthonormal row vectors. The polar decomposition for rectangular matrices finds the closest point on the Stiefel manifold to a given matrix, in the Frobenius norm. This decomposition is the rectangular analogue of the well-known polar decomposition for square matrices.

11. Complexity Analysis

The proposed NRS*f*M solvers achieves its efficiency primarily by replacing optimization heavy procedures, i.e., semidefinite programming (SDP) for rotation estimation and alternating-direction methods (ADMM) for shape estimation, with feed-forward modules composed only of basic linear algebra primitives (matrix multiplications and SVDs). In this section, we analyze the computational and memory complexity of both stages and contrast them with the classical baselines.

11.1. Rotation Estimation

Let $\mathbf{W} \in \mathbb{R}^{2F\times P}$ be the measurement matrix with F denoting the number of frames. The popular SDP-based Euclidean correction involves solving, for each basis triplet k, a SDP of the form

$$\min_{\Psi_k \succeq 0} \ \mathrm{trace}(\Psi_k) \ \text{ s.t. } \mathbf{A}\mathrm{vec}(\Psi_k) = 0; \mathrm{rank}(\Psi_k) \leq 3,$$

where A encodes the per-frame orthonormality constraints and $\Psi_k \in \mathbb{R}^{3K\times 3K}$. Even though the SDP is "small and fixed" in size ($(2K^2-K)$ constraints—refer [15]), each solve involves high time and memory complexity because originally proposed SDP solvers rely on interior-point methods whose cost scales cubically with the matrix dimension of Ψ_k and quadratically with the number of constraints. An interior-point SDP solver involves Cholesky or eigen-decomposition of matrices of size $3K$, and each iteration requires $O((3K)^6)$ time and $O((3K)^4)$ memory. Although generally $K \in [3, 13]$ is small, this fixed overhead becomes dominant as F or P grow, since the SDP must be solved once per sequence or per re-estimation. Furthermore, SDP solvers are CPU-bound and difficult to parallelize.

Proposed Euclidean Correction. Our approach to Euclidean correction overcomes the SDP difficulties with a learnable corrective factor representation. Each basis triplet is parameterized as $\mathbf{Y}_k \in \mathbb{R}^{3K\times 3}$ (`nn.Parameter` in PyTorch), and the associated Gram matrix $\Psi_k = \mathbf{Y}_k\mathbf{Y}_k^T$ automatically satisfies the positive semidefinite (PSD) and at most rank-3 constraints by construction. For each k, we compute $\hat{\Pi}\mathbf{Y}_k \in \mathbb{R}^{2F\times 3}$, split this matrix into F double rows $\{\mathbf{\Omega}_{ik} = [\hat{\pi}_{2i-1}\mathbf{Y}_k; \hat{\pi}_{2i}\mathbf{Y}_k]\}_{i=1}^F$, and evaluate the orthogonality residuals

$$\varepsilon_{ik}^{(1)} = (\|\mathbf{g}_{1,ik}\|_2^2 - \|\mathbf{g}_{2,ik}\|_2^2)^2; \ \ \varepsilon_{ik}^{(2)} = (\mathbf{g}_{1,ik}\cdot\mathbf{g}_{2,ik})^2,$$

which constitute the orthogonality loss

$$\mathcal{L}_{\text{orth}} = \frac{1}{FK}\sum_{i,k}(\varepsilon_{ik}^{(1)} + \varepsilon_{ik}^{(2)}). \tag{28}$$

Backpropagation through $\hat{\Pi}\mathbf{Y}_k$ updates $\mathbf{Y}_k$ via simple general matrix–matrix multiplication (GEMM).

Per-iteration complexity. The dominant operation is the matrix multiplication $\hat{\Pi}\mathbf{Y}_k$ for each k, whose cost is

$$O\big((2F)(3K)\cdot 3\big) = O(18FK) \ \text{ flops.}$$

Since there are K such bases, the overall cost per forward or backward pass is $O(FK^2)$ time and $O(FK+K^2)$ memory. This cost is quadratic in K with a very small constant factor (tens of floating-point operations per entry). The per-frame row-polar projection $\mathbf{R}_i = (\mathbf{\Omega}_{ik}\mathbf{\Omega}_{ik}^T)^{-1/2}\mathbf{\Omega}_{ik}$ is $O(F)$ and negligible. All operations are differentiable and parallelizable over k and i.

Comparison to SDP Modeling. Interior-point SDP solvers can scale poorly with the matrix dimension. In practice, even for small K, their fixed overhead and CPU-bound implementation dominate the rotation stage. While the classical SDP has superlinear dependence[2] on K and is bound by dense CPU linear algebra, our rotation solver is dominated by a few GEMMs of size $(2F\times 3K)\times(3K\times 3)$, which execute efficiently on modern accelerators. Theoretical runtime improves from $O(K^6)$ per sequence to $O(FK^2)$ and the memory footprint from $O(K^4)$ to $O(FK)$. Empirically, on standard sequences ($F \approx 300, K \leq 13$) the rotation recovered using our solver is order of magnitude faster while yielding equivalent orthonormality accuracy.

11.2. Shape Estimation

Classical ADMM-based solvers. Given $\mathbf{R}$, the shape $\mathbf{S}$ is recovered by minimizing

$$\min_{\mathbf{S}} \ 0.5\|\mathbf{M}\odot(\mathbf{W}-\mathbf{R}\mathbf{S})\|_F^2 + \|\mathbf{S}^\sharp\|_{*,w}. \tag{29}$$

To solve the above optimization via ADMM requires introduction of an auxiliary variable $\mathbf{U} = \mathbf{S}^\sharp$ and a dual variable $\mathbf{L}$ yielding the following iterations:

$$\begin{aligned}\mathbf{S}^{k+1} := \arg\min_S \|\mathbf{M}\odot(\mathbf{W}-\mathbf{R}\mathbf{S})\|_F^2 \ + \\ \tfrac{\rho}{2}\|g(\mathbf{S}) - \mathbf{U}^k + \mathbf{L}^k/\rho\|_F^2\end{aligned} \tag{30}$$

$$\mathbf{U}^{k+1} := \arg\min_{\mathbf{U}} \|\mathbf{U}\|_{*,w} + \tfrac{\rho}{2}\|\mathbf{U} - g(\mathbf{S}^{k+1}) - \mathbf{L}^k/\rho\|_F^2 \tag{31}$$

$$\mathbf{L}^{k+1} := \mathbf{L}^k + \rho(g(\mathbf{S}^{k+1}) - \mathbf{U}^{k+1}). \tag{32}$$

The $\mathbf{S}$ update rrequires solving a large linear system $(\mathbf{R}^T\mathbf{R}+\rho\mathbf{I})\mathbf{S} = \mathbf{R}^T\mathbf{W} + \rho(g^{-1}(\mathbf{U}) - \mathbf{Y}/\rho)$, which for a dense $\mathbf{R}$ costs $O((3F)^3P)$ or, at best, $O(FP)$ if block-diagonal structure is exploited. The $\mathbf{U}$ update involves a full SVD of an $F\times 3P$ matrix $O(F(3P)^2)$, and the dual update is linear. With N_{ADMM} iterations, total runtime scales as

$$O\Big(N_{\text{ADMM}}\big(F(3P)^2 + (3F)^3P\big)\Big),$$

[2] A superlinear function grows more rapidly than a linear function for large values of its input.

and memory $O(FP)$ for the primal variables plus $O(FP)$ for the duals.

Proposed solver for shape estimation. Our unrolled layer eliminates the dual variable and linear system. Each layer $k = 0, \ldots, N_k - 1$ performs: ***(i)*** gradient of the reprojection term: $\mathbf{G} = \mathbf{R}^T(\mathbf{M} \odot (\mathbf{RS} - \mathbf{W}))$, cost $O(FP)$, ***(ii)*** reshuffled shape domain update: $\mathbf{Z}^\sharp = g(\mathbf{S}) - \alpha g(\mathbf{G})$, ***(iii)*** WSVT: $\mathbf{X}^\sharp = \mathcal{S}_{\tau_k, w}(\mathbf{Z}^\sharp)$, cost $O(\min(F, 3P)^2 \max(F, 3P))$, and ***(iv)*** reshape back: $\mathbf{S} \leftarrow g^{-1}(\mathbf{X}^\sharp)$.

For a general sparse NRS*f*M case $F > 3P$, i.e., many frames and moderate points, and so SVD dominates with cost $O(FP^2)$ per layer. Thus per-layer complexity is $O(FP^2)$ time and $O(FP)$ memory. In dense settings where $3P \gg F$, the cost is $O(F^2P)$. Thus, the total shape-stage cost is $O(N_k FP^2)$ when $F > 3P$, and $O(N_k F^2 P)$ when $3P > F$. FISTA acceleration and continuation scheduling add only $O(FP)$ bookkeeping.

All in all, if we compare our approach to ADMM based optimization then (a) ADMM's linear solves, i.e., $(\mathbf{R}^T\mathbf{R} + \rho\mathbf{I})^{-1}$ inversion is the major bottleneck, both computationally and numerically, is removed entirely, (b) Dual variables are absent in our formulation, significantly reducing the memory footprint, (c) our deterministic solver typically converges in $N_k \approx$ 40-60 layers, whereas ADMM requires hundreds of iterations with penalty tuning, and (d) each layer uses only GEMM and SVD, which are the most optimized primitives in BLAS and GPU libraries.

Asymptotically, both ADMM and our solver require SVDs of the same shape matrix, yet ADMM incurs additional $O((3F)^3 P)$ linear solves per iteration, making it 1-2 orders of magnitude slower. Empirically, our unrolled solver achieves identical objective values with less runtime and lower memory overhead, specifically for dense datasets.

11.3. Discussion

For typical benchmarks ($F \approx 300$, $P \approx 30 - 100$, $K \leq 13$), the proposed approach runs in a few seconds on NVIDIA A6000 GPU or Apple M3 processor, compared to several minutes for SDP/ADMM pipelines on CPU/GPU's. Our introduced improvement transforms NRS*f*M matrix factorization approach from a slow, research-oriented method into a practical and scalable tool for non-rigid deformable object 3D reconstruction so that it can further be used in downstream computer vision tasks which at present is mostly machine-learning driven. Table 13 summarizes the asymptotic complexity comparison of both SDP/ADMM based SOTA methods with our proposed NRS*f*M approach.

12. Lipschitz properties: Re-projection term

In our formulation, the overall objective is composed of $f(\mathbf{S})$ and $g(\mathbf{S})$, where

$$f(\mathbf{S}) = 0.5 * \|\mathbf{M} \odot (\mathbf{RS} - \mathbf{W})\|_F^2, \text{ with } \mathbf{S} \in \mathbb{R}^{3F \times P},$$

It enforces re-projection consistency between the estimated 3D structure $\mathbf{S}$ and the observed 2D points $\mathbf{W}$. In our formulation, $g(\mathbf{S})$ is the regularization, which is the non-smooth part that promotes the low-rank structure across frames. Now to apply proximal gradient method, we need to take gradient step on $f()$ and the proximal step on $g()$. For that gradient step to be valid and stable, the gradient of $f()$ must be Lipschitz continuous for some constant $L > 0$ (Lipschitz constant). This condition tells us that the gradient of f does not change too abruptly and geometrically the curvature of f is bounded.

Next, we will show step by step that the smooth part of the our NRS*f*M objective is ***(i)*** continuously differentiable with an explicit gradient, and ***(ii)*** has a globally Lipschitz-continuous gradient, i.e., f is L–smooth with

$$L \leq \|\mathbf{R}\|_2^2,$$

where, $\|\cdot\|_2$ denotes the spectral norm and under orthography $\|\mathbf{R}\|_2 \leq 1$.

12.1. Preliminaries and notations

Hilbert space: Assume V, a vector space that is metrically *complete* with respect to the metric induced by its own norm. If V's norm induces a complete metric space and is itself induced by an inner product on V, we say V is a Hilbert space [1, 65]. Here, the notion of Hilbert space $\mathbb{R}^{m \times n}$ is used with the Frobenius inner product $\langle \mathbf{X}, \mathbf{Y} \rangle = \mathbf{Tr}(\mathbf{X}^T\mathbf{Y})$ and the induced Frobenius norm $\|\mathbf{X}\|_F = \sqrt{\langle \mathbf{X}, \mathbf{X} \rangle}$.

As defined in the main paper, let $\mathbf{R} \in \mathbb{R}^{2F \times 3F}$ be the block-diagonal rotation matrix, $\mathbf{M} \in \{0, 1\}^{2F \times P}$ a fixed binary visibility mask and $\mathbf{W} \in \mathbb{R}^{2F \times P}$ the measurement matrix. Let's define the linear operators

$$\mathcal{H}_\mathbf{M}(\mathbf{Z}) \triangleq \mathbf{M} \odot \mathbf{Z} \quad \text{(Hadamard masking)}, \quad \mathcal{R}(\mathbf{S}) \triangleq \mathbf{RS}.$$

We will use of the following facts: ***(i)*** $\mathcal{H}_\mathbf{M}$ is an orthogonal projection: $\mathcal{H}_\mathbf{M} = \mathcal{H}_\mathbf{M}^* = \mathcal{H}_\mathbf{M}^2$ and $\|\mathcal{H}_\mathbf{M}(\mathbf{Z})\|_F \leq \|\mathbf{Z}\|_F$ (i.e., $\|\mathcal{H}_M\|_{op} = 1$). ***(ii)*** $\mathcal{R}$ is bounded with $\|\mathcal{R}\|_{op} = \|\mathbf{R}\|_2$, and $\mathcal{R}^*(\mathbf{T}) = \mathbf{R}^T\mathbf{T}$. For convenience, lets define the composite linear operator

$$\mathcal{A} : \mathbb{R}^{3F \times P} \to \mathbb{R}^{2F \times P}, \quad \mathcal{A}(\mathbf{S}) = \mathcal{H}_\mathbf{M}(\mathcal{R}(\mathbf{S})) = \mathbf{M} \odot (\mathbf{RS}),$$

and its adjoint $\mathcal{A}^*(\mathbf{T}) = \mathcal{R}^*(\mathcal{H}_\mathbf{M}(\mathbf{T})) = \mathbf{R}^T(\mathbf{M} \odot \mathbf{T})$. Using ***(i)*** and ***(ii)***,

$$\|\mathcal{A}\|_{op} \leq \|\mathcal{H}_\mathbf{M}\|_{op} \|\mathcal{R}\|_{op} = \|\mathbf{R}\|_2. \tag{33}$$

12.2. Differentiability and the gradient of f

We rewrite f in operator form:

$$f(\mathbf{S}) = 0.5 * \|\mathcal{A}(\mathbf{S}) - \mathbf{B}\|_F^2, \quad \mathbf{B} \triangleq \mathcal{H}_\mathbf{M}(\mathbf{W}) = \mathbf{M} \odot \mathbf{W}.$$

Stage	Classical (SDP/ADMM)	Ours
Rotation estimation	$O(K^6)$ per sequence (SDP)	$O(FK^2)$
Shape estimation (per iter.)	$O(F(3P)^2) + (3F)^3P$	$O(FP^2)$ if $F > 3P$, and $O(F^2P)$ if $3P > F$
Linear solves / duals	required	none
Hardware acceleration	NA (CPU-bound)	excellent
Scales with frames F	superlinear	Regime-dependent: linear in F, if $F > 3P$ else quadratic
Scales with points P	quadratic	Regime-dependent: linear in P, if $3P > F$ else quadratic
Scales with bases K	superlinear (K^6)	quadratic (K^2)

Table 13. Asymptotic complexity comparison between SOTA SDP and ADMM-based NRS*f*M and the proposed approach.

Using standard rules for least squares in Hilbert spaces[3],

$$\nabla f(\mathbf{S}) = \mathcal{A}^*\big(\mathcal{A}(\mathbf{S}) - \mathbf{B}\big) = \mathbf{R}^T\Big(\mathbf{M} \odot \big(\mathbf{R}\mathbf{S} - \mathbf{W}\big)\Big).$$

Thus f is (globally) $\mathcal{C}^1$ with affine gradient $\nabla f(\mathbf{S}) = \mathcal{A}^*\mathcal{A}(\mathbf{S}) - \mathcal{A}^*\mathbf{B}$.

12.3. Lipschitz continuity of the gradient (L–smoothness)

We say f satisfies the Lipschitz condition [44] on a set D if there is a constant $L \geq 0$ such that

$$|f(t,u) - f(t,v)| \leq L|u - v| \;\; \forall (t,u),(t,v) \in D.$$

We now show that ∇f is globally Lipschitz and compute an admissible constant. For any $\mathbf{S}_1, \mathbf{S}_2 \in \mathbb{R}^{3F\times P}$,

$$\begin{aligned}\nabla f(\mathbf{S}_1) - \nabla f(\mathbf{S}_2) &= \mathcal{A}^*\big(\mathcal{A}(\mathbf{S}_1) - \mathbf{B}\big) - \mathcal{A}^*\big(\mathcal{A}(\mathbf{S}_2) - \mathbf{B}\big) \\ &= \mathcal{A}^*\mathcal{A}\left(\mathbf{S}_1 - \mathbf{S}_2\right).\end{aligned}$$

Taking norms and using sub-multiplicativity of the operator norm,

$$\begin{aligned}\|\nabla f(\mathbf{S}_1) - \nabla f(\mathbf{S}_2)\|_F &\leq \|\mathcal{A}^*\|_{op}\|\mathcal{A}\|_{op}\|\mathbf{S}_1 - \mathbf{S}_2\|_F \\ &= \|\mathcal{A}\|_{op}^2\,\|\mathbf{S}_1 - \mathbf{S}_2\|_F,\end{aligned} \tag{34}$$

since $\|\mathcal{A}^*\|_{op} = \|\mathcal{A}\|_{op}$ in Hilbert spaces. Combining (34) with (33), we obtain the global Lipschitz bound

$$\|\nabla f(\mathbf{S}_1) - \nabla f(\mathbf{S}_2)\|_F \;\leq\; \|\mathbf{R}\|_2^2\,\|\mathbf{S}_1 - \mathbf{S}_2\|_F, \;\forall\, \mathbf{S}_1, \mathbf{S}_2,$$

i.e., f is L–smooth with any $L \geq \|\mathbf{R}\|_2^2$. Under orthographic projection, R has orthonormal rows blockwise and $\|\mathbf{R}\|_2 \leq 1$, hence one may take $L = 1$. The above single inequality is the Lipschitz-gradient proof.

Alternative proof (direct). Without operator notation, let $\Delta\mathbf{S} = \mathbf{S}_1 - \mathbf{S}_2$. Then

$$\nabla f(\mathbf{S}_1) - \nabla f(\mathbf{S}_2) = \mathbf{R}^T\big(\mathbf{M} \odot (\mathbf{R}\Delta\mathbf{S})\big).$$

[3]Linear least squares in a Hilbert space extends the concept of minimizing the squared error to infinite-dimensional vector spaces, where the solution is found by finding a vector that is orthogonal to the error space.

Using the contractivity $\|\mathbf{M} \odot \mathbf{X}\|_F \leq \|\mathbf{X}\|_F$ for Hadamard masking and sub-multiplicativity,

$$\begin{aligned}\|\nabla f(\mathbf{S}_1) - \nabla f(\mathbf{S}_2)\|_F &\leq \|\mathbf{R}^T\|_2\|\mathbf{M} \odot (\mathbf{R}\Delta\mathbf{S})\|_F \\ &\leq \|\mathbf{R}\|_2\|\mathbf{R}\Delta\mathbf{S}\|_F \leq \|\mathbf{R}\|_2^2\|\Delta\mathbf{S}\|_F,\end{aligned} \tag{35}$$

which is the same bound.

12.4. On Lipschitz continuity of f itself

A globally Lipschitz function h satisfies $|h(\mathbf{X}) - h(\mathbf{Y})| \leq K\|\mathbf{X} - \mathbf{Y}\|$ for some K independent of $\mathbf{X}, \mathbf{Y}$. Quadratic functions like $f(\mathbf{S}) = \frac{1}{2}\|\mathcal{A}(\mathbf{S}) - \mathbf{B}\|_F^2$ are in general not globally Lipschitz on an unbounded domain as they grow quadratically, but they are Lipschitz on any bounded set. For our case $\mathbf{S}_1, \mathbf{S}_2$ can safely be assumed in a finite ball $\{\mathbf{S} : \|\mathbf{S}\|_F \leq r \text{ (radius)}\}$, the mean value theorem in Hilbert spaces gives

$$\begin{aligned}&|f(\mathbf{S}_1) - f(\mathbf{S}_2)| \\ &\leq \sup_{t\in[0,1]} \|\nabla f(\mathbf{S}_2 + t(\mathbf{S}_1 - \mathbf{S}_2))\|_F\,\|\mathbf{S}_1 - \mathbf{S}_2\|_F \\ &\leq \Big(\|\mathcal{A}\|_{op}^2 r + \|\mathcal{A}\|_{op}\,\|\mathbf{B}\|_F\Big)\|\mathbf{S}_1 - \mathbf{S}_2\|_F\end{aligned} \tag{36}$$

so f is Lipschitz on bounded sets with a constant that depends linearly on the radius r. For algorithmic purposes (step-size selection, convergence of proximal-gradient), the relevant property is the Lipschitz continuity of the gradient established above.

12.5. Consequences (step sizes and descent)

Since f has an L–Lipschitz gradient with $L \leq \|\mathbf{R}\|_2^2$, any fixed step $\alpha \in (0, 1/L]$ guarantees that the standard descent lemma holds:

$$f(\mathbf{S} + \Delta) \;\leq\; f(\mathbf{S}) + \langle \nabla f(\mathbf{S}), \Delta\rangle \;+ \tfrac{L}{2}\|\Delta\|_F^2, \;\; \forall\, \mathbf{S}, \Delta.$$

This is the condition needed to ensure monotonic decrease in the proximal-gradient (and FISTA-with-restart [64]) iterations used in our sharp-domain spectral–proximal solver. In particular, under orthography one may safely choose $\alpha \leq 1$.

Q. Why use the notion of Hilbert space to establish the last proof (Sec.12)? Such an abstract functional analysis to prove this is optional, but treating matrices as elements of a Hilbert space with the Frobenius inner product makes the proof short, clean, and modular. Furthermore, in a Hilbert space every bounded linear operator $\mathcal{A}$ has a unique adjoint $\mathcal{A}^*$ defined by

$$\langle \mathcal{A}(\mathbf{X}), \mathbf{Y} \rangle = \langle \mathbf{X}, \mathcal{A}^*(\mathbf{Y}) \rangle \; \forall \, \mathbf{X}, \mathbf{Y} \text{ and, } \|\mathcal{A}\|_{op} = \|\mathcal{A}^*\|_{op}.$$

This lets us write gradients and Lipschitz bounds using operator norms instead of componentwise calculations. Other benefits it brings which is specific to the form proposed in the NRS*f*M formulation are: (a) Clean expression for the gradient, i.e., $\nabla f(\mathbf{S}) = \mathcal{A}^*\big(\mathcal{A}(\mathbf{S}) - \mathbf{B}\big)$ follows from the Reisz representation in a Hilbert space [1], (b) Immediate Lipschitz bound, i.e., $\|\nabla f(\mathbf{S}_1) - \nabla f(\mathbf{S}_2)\|_F = \|\mathcal{A}\|_{op}^2 \, \|\mathbf{S}_1 - \mathbf{S}_2\|_F$. The only thing to upper bound is $\|\mathcal{A}\|$, which we establish by $\|\mathbf{R}\|_2$ due to $\|\mathcal{H}_{\mathbf{M}}\| \leq 1$, and lastly (c) if we change the mask $\mathbf{M}$ or the camera model $\mathbf{R}$ the same expression could work, we just need to update the $\|\mathcal{A}\|$ norm.

Long story short, the Hilbert-space view lets us treat NRS*f*M re-projection map as a single bounded linear operator. After that all the usual least-squares calculus and norm inequalities apply verbatim.

13. Additional Discussion and Results

1. Sequence-wise accuracy tradeoffs. We use a sequential rotation-then-shape design to preserve the main goal of the solver. A coordinate-descent refinement that alternates between camera and shape updates may further improve accuracy on some sequences, especially when the initial camera estimate is imperfect, but it would reintroduce an outer iterative loop and increase runtime. We therefore treat such refinement as an optional accuracy-oriented extension rather than part of the fast no-cone, no-dual solver.

2. Accuracy limit: coordinate descent. The sequence-wise behavior of our apporach is consistent with the priors used by competing methods. Our solver performs strongly when the factor-side rotation estimate is stable and the low-rank shape-space prior is sufficient, e.g., Walking and Stretch. Larger gaps appear on sequences such as Drink, Dance, and Shark, where methods such as TSM and OPM benefit from additional temporal, Procrustean, rotation-averaging, or deformation priors. Our method deliberately avoids these priors to keep the solver fast and broadly applicable, which explains the observed accuracy-runtime tradeoff.

3. Stage-wise runtime accounting. To clarify the runtime reported in the main paper, Table 14 separates the cost of the two solver stages on the CMU MoCap sequences and dense face benchmarks. The rotation time corresponds to optimizing the factor-side Euclidean correction and applying the row-polar projection to recover the block-orthographic camera matrix. The shape time corresponds to the fixed-depth no-dual spectral-proximal solver after the rotations are fixed. The total runtime is the sum of these two stages. In particular, longer, denser, and challenging sequences such as Drink, Dance, Shark, and Dense Face Sequence incur higher cost, but the total runtime remains within a practical range.

Dataset	Rotation time	Shape time	Total time
CMU MoCap Sequence			
Drink	8.51	6.45	14.96
Pickup	7.79	4.88	12.67
Yoga	7.73	4.19	11.92
Stretch	7.83	4.21	12.04
Dance	8.57	6.39	14.96
Face	8.42	4.66	13.08
Walking	8.52	4.84	13.36
Shark	8.27	9.51	17.78
Dense Face Sequence			
Face Seq.1	1.24	7.05	8.29
Face Seq.2	1.25	6.93	8.18
Face Seq.3	1.73	13.28	15.01
Face Seq.4	1.65	13.51	15.16

Table 14. Stage-wise runtime of our solver on sparse MoCap and dense face benchmarks. Rotation time includes factor-side Euclidean correction and row-polar projection. Shape time includes the fixed-depth no-dual spectral-proximal solver after rotations are fixed. Total time is the combined solver runtime. All times are reported in seconds.

4. Sensitivity to the shape-space rank K. Table 15 shows the result on the standard CMU MoCap sequences with different K values. We vary $K \in \{10, 11, 12, 13\}$. *The results show that the proposed solver is not fragile to a narrowly tuned rank value*. The overall trend remains stable and does not show a monotonic advantage from simply increasing model capacity. In several cases, nearby K values slightly improve the reported error.

Sequence	$K = 10$	$K = 11$	$K = 12$	$K = 13$
Drink	0.0161	<u>0.0166</u>	0.0164	**0.0153**
Pickup	0.0272	**0.0239**	<u>0.0267</u>	0.0256
Yoga	0.0257	**<u>0.0148</u>**	0.0236	0.0207
Stretch	0.0122	**0.0118**	0.0119	<u>0.0128</u>

Table 15. Sensitivity of e_{3d} to the shape-space rank K on CMU MoCap sequences. The underlined entries indicate the K values used in the main-paper comparison following prior shape-space baselines, while bold entries indicate the best result in the sweep. The results show that the proposed solver is stable across nearby ranks and is not dependent on aggressive per-sequence rank tuning.

5. Sensitivity to continuation and spectral safeguards. Tables 16-18 study the effect of the continuation and spectral-safeguard parameters in the unrolled shape solver. Each

$\eta \downarrow$ /cap $\rightarrow$	0.90	0.95	0.99
1.02	(0.0660, 0.0211)	(0.0108, 0.0200)	(0.0112, 0.0177)
1.05	(0.1194, 0.0271)	(0.0640, 0.0265)	(0.0168, 0.0267)
1.10	(0.1369, 0.0577)	(0.1437, 0.0592)	(0.1248, 0.0591)

Table 16. $(e_{3d}^{\text{Drink}}, e_{3d}^{\text{Pickup}})$ with $N_{\text{pres}} = 0$

$\eta \downarrow$ /cap $\rightarrow$	0.90	0.95	0.99
1.02	(0.0510, 0.0233)	(**0.0108**, 0.0204)	(0.0111, **0.0176**)
1.05	(0.1219, 0.0280)	(0.0618, 0.0264)	(0.0166, 0.0267)
1.10	(0.1360, 0.0698)	(0.1411, 0.0621)	(0.1234, 0.0590)

Table 17. $(e_{3d}^{\text{Drink}}, e_{3d}^{\text{Pickup}})$ with $N_{\text{pres}} = 1$

$\eta \downarrow$ /cap $\rightarrow$	0.90	0.95	0.99
1.02	(0.0976, 0.0238)	(0.0108, 0.0224)	(0.0113, 0.0186)
1.05	(0.1496, 0.0261)	(0.1205, 0.0274)	(0.0249, 0.0268)
1.10	(0.1690, 0.0880)	(0.1658, 0.0820)	(0.1570, 0.0826)

Table 18. $(e_{3d}^{\text{Drink}}, e_{3d}^{\text{Pickup}})$ with $N_{\text{pres}} = 2$

entry reports $(e_{3d}^{\text{Drink}}, e_{3d}^{\text{Pickup}})$, while varying the number of preserved singular values N_{pres}, the continuation rate η, and the shrinkage cap. All other parameters are kept fixed. The default setting used in the main paper is $N_{\text{pres}} = 1$, $\eta = 1.05$, and $\text{cap} = 0.99$.

The results show that the solver behavior is predictable rather than arbitrary. The continuation rate η has the strongest effect: a large value such as $\eta = 1.10$ relaxes the shrinkage too quickly and consistently degrades reconstruction accuracy. Slower or moderate continuation, $\eta \in \{1.02, 1.05\}$, is substantially more stable. The number of preserved modes also has the expected effect, i.e., preserving no mode or only the dominant mode gives similar performance in stable regimes, while preserving two modes can weaken low-rank regularization and increase error, especially on Drink. Finally, caps in the range 0.95-0.99 are more reliable than 0.90, suggesting that overly restrictive shrinkage caps can prevent effective spectral regularization.

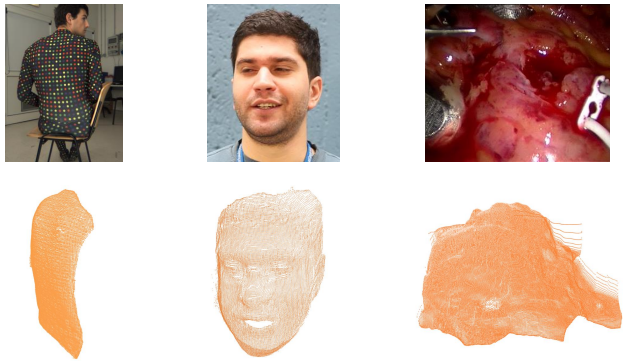
Importantly, the main-paper default setting on these two sequences gives stable results. Yet, aggressively striving for best possible (η, cap) may further improve results. For instance, some $\eta = 1.02$ settings yield slightly lower errors. *Thus, the reported main results are not obtained by aggressive per-sequence tuning*. Instead, **the default parameters are used as a single fixed setting across experiments, providing a stable accuracy-runtime tradeoff while preserving the intended no-cone no-dual solver design.**

6. Pose-impact diagnostic using GT rotations. To isolate the influence of camera-pose estimation on final shape recovery, Table 19 compares the proposed shape solver using our estimated rotations against the same solver using ground-truth rotations on MoCap sequences where GT rotations are available. This diagnostic follows the same spirit as prior shape-space NRS*f*M analyses. In particular, Kumar and Van Gool [33] evaluated rotation accuracy using GT rotations and observed that, under the factorization model $\mathbf{W} = \mathbf{RS}$,

Sequence	Ours $\mathbf{R}$	GT $\mathbf{R}$	Gap
Drink	0.0166	0.0158	0.0008
Pickup	0.0267	0.0237	0.0030
Yoga	0.0148	0.0148	0.0000
Stretch	0.0127	0.0128	0.0001

Table 19. Pose-impact diagnostic on MoCap sequences with ground-truth rotations. We compare the 3d reconstruction results using estimated rotation and ground-truth rotation, respectively. The gap is computed as $\text{Gap} = |e_{3d}(\text{Ours } \mathbf{R}) - e_{3d}(\text{GT } \mathbf{R})|$.

even ground-truth rotations do not by themselves guarantee ground-truth shape recovery. Our results are consistent with this observation. Replacing our estimated rotations with GT rotations changes e_{3d} only marginally: 8×10^{-4} on Drink, 3×10^{-3} on Pickup, 0 on Yoga, and 10^{-4} on Stretch. Thus, the factor-side Euclidean correction provides camera estimates accurate enough for downstream shape recovery, and the remaining error is largely governed by the low-rank shape prior, deformation complexity, and the intended speed-accuracy tradeoff of the no-dual solver rather than gross pose-estimation gap.



(a) Back Sequence (b) Face Sequence (c) Heart Sequence

Figure 10. Qualitative results on real dense dataset sequence [18].

7. Qualitative evaluation on real dense sequences. We further evaluate our approach on the real dense NRS*f*M dataset [18, 19]. This dataset consists of monocular videos captured in natural settings with varying illumination and large non-rigid deformation. It contains three sequences namely Face, Back, and Heart, with 120, 150, and 80 frames, respectively. The dataset provides dense 2D feature tracks for all three sequences, with $28,332$ tracks for Face, $20,561$ tracks for Back, and $68,295$ tracks for Heart without GT 3D shape. Figure 10 shows using our approach, demonstrating visually coherent dense 3D structure from real monocular videos.

Additional Visualization. Figure 11, Figure 12, Figure 13, and Figure 14 provide additional qualitative results on H3WB [57], NRS*f*M challenge benchmark [26], CMU Mocap [5], and Dense Face Dataset [18], respectively.

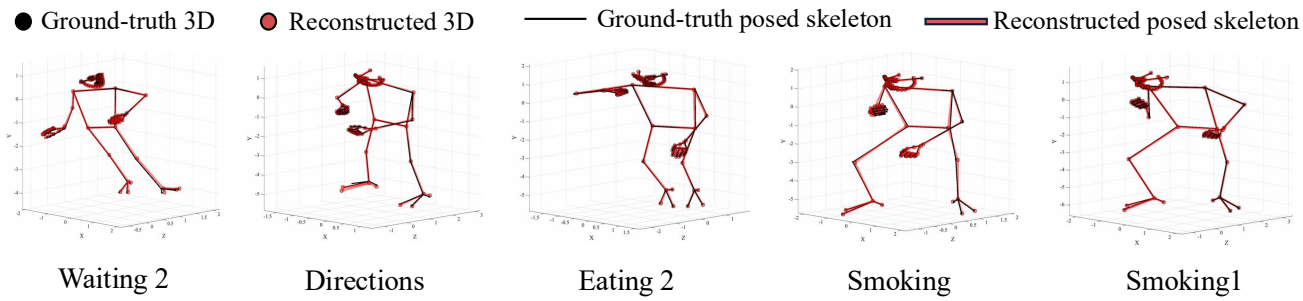


Figure 11. Qualitative results on different categories present in H3WB dataset [57].

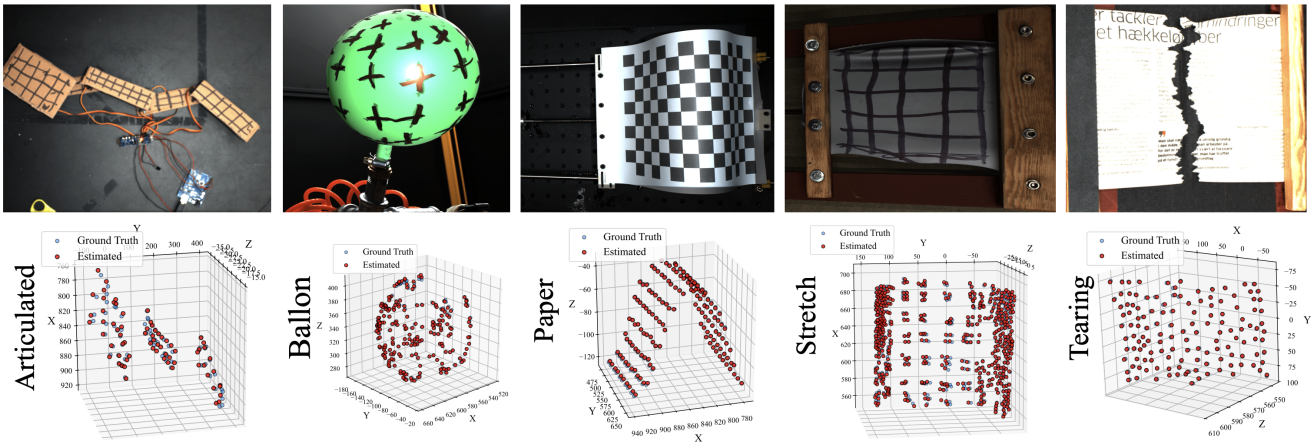


Figure 12. Qualitative results on different categories in NRS*f*M challenge dataset [26].

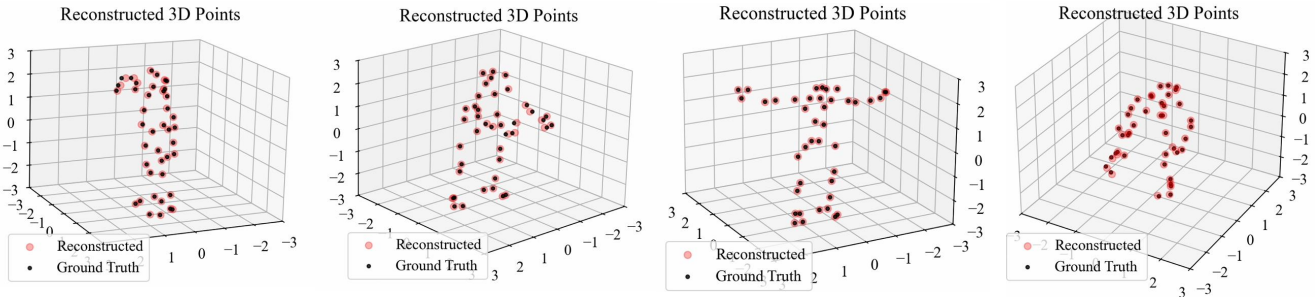


Figure 13. Qualitative results on CMU Mocap dataset [5].

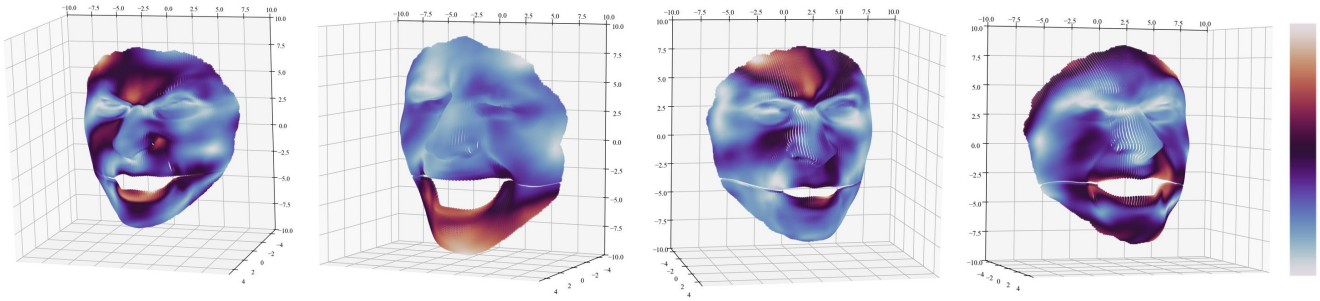


Figure 14. Qualitative results on Dense NRS*f*M Face dataset sequence [18] showing the 1st order deformation field.